97.1
Pass

Ravindra L Patil

PDF generated at: 12 Feb 2026 09:04:25 UTC

View this report on HackerRank

Score

97.1% • 170.0 / 175
scored in eClerx - Java Coding Test - 3+ Years New in 23 min 1 sec on 12 Feb 2026 14:09:31 IST

Candidate Information

Email	ravindralpatil726@gmail.com
Test	eClerx - Java Coding Test - 3+ Years New
Candidate Packet	View
Taken on	12 Feb 2026 14:09:31 IST
Time taken	23 min 1 sec/ 90 min
Work Experience	> 5 years
Invited by	Nancy
Invited on	12 Feb 2026 13:55:50 IST

Skill Distribution

No.	Skill	Score
1	Spring Boot Advanced	50%

2	Spring Boot Intermediate	100%	
3	Problem Solving Intermediate	100%	
4	Java Intermediate	100%	

Tags Distribution

Spring Boot	75%	Hard	50%
Medium	100%	Spring	100%
Exception Handling	100%	Aspect-Oriented Programming	100%
Graphs	100%	Greedy	100%
Java	100%		

Questions

Spring Boot (Advanced) • 20 / 25

No.	Question	Skill	Score	Code Quality	Time Taken	Status
1	JSON Deserialization Multiple Choice	Spring Boot (Advanced)	0/5	-	35 sec	⊗

2	Specification Multiple Choice	Spring Boot (Advanced)	5/5	-	19 sec	✓
3	Security Custom Login Multiple Choice	Spring Boot (Intermediate)	5/5	-	22 sec	✓
4	Spring Scopes Multiple Choice	Spring Boot (Intermediate)	5/5	-	36 sec	✓
5	Predict the Output Multiple Choice	Spring Boot (Intermediate)	5/5	-	30 sec	✓

Problem Solving (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
6	Minimize Path Value Coding	Problem Solving (Intermediate)	75/75	-	12 min 16 sec	✓

Java (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
7	Java: Optimized Sequential Video Watching Coding	Java (Intermediate)	75/75	-	8 min 1 sec	✓

Candidate Feedback

Good

1. JSON Deserialization

Incorrect

Multiple Choice | Spring Boot | Hard

Question description

CheckerCreateRequest.java

```
// Annotation here
public class CheckerCreateRequest {
    private String name;
    private String surname;
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getSurname() {
        return surname;
    }
    public void setSurname(String surname) {
        this.surname = surname;
    }
}
```

CheckerController.java

```
@RestController
@RequestMapping("/checkers")
public class CheckerController {

    @Autowired
    CheckerService checkerService;

    @PostMapping
    public ResponseEntity createChecker(@RequestBody CheckerCreateRequest checkerCreateReque
st) {
```

```
        checkerService.createChecker(checkerCreateRequest);
        return new ResponseEntity(HttpStatus.CREATED);
    }
}
```

Request Json data

```
{
    "checker":{
        "name": "Checker",
        "surname":"Number2"
    }
}
```

A POST request with "Request Json data" is made to "/checkers". What annotation needs to be added to the "CheckerCreateRequest" class so that it can successfully deserialize and process the request?

Interviewer guidelines

@JsonRootName doesn't add checker as a wrapper attribute by itself. To make it successfully deserialize with given json annotations on D should be used together.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ● **@JsonRootName("checker")**
- ○ @JsonName("checker")
- ○ @JsonTypeName("checker")

@JsonTypeInfo(include = JsonTypeInfo.As.WRAPPER_OBJECT, use = JsonTypeInfo.Id.NAME)
@JsonTypeName("checker")

No comments.

2. Specification

Correct

Multiple Choice | Spring Boot | Hard

Question description

Which of the following options can be used for dynamic jpa queries with spring-data-jpa repositories?

Candidate's Solution

Options: (Expected answer indicated with a tick)

- PreparedStatement
- **Specification**
- Criteria
- Query

No comments.

3. Security Custom Login

Correct

Multiple Choice | Spring Boot | Medium

Question description

Which of the following options can be used to create a custom login page in Spring Security?

Candidate's Solution

Options: (Expected answer indicated with a tick)

- protected void configure(HttpSecurity http) throws Exception { http .authorizeRequests() .anyRequest().authenticated() .and() .formLogin() .and() .httpBasic(); }
- protected void configure(HttpSecurity http) throws Exception { http .authorizeRequests() .anyRequest().authenticated() .and() .formLogin() .loginPage("/login"); }
- **protected void configure(HttpSecurity http) throws Exception { http .authorizeRequests() .anyRequest().authenticated() .and() .formLogin() .loginPage("/login") .permitAll(); }**
- None of the above

No comments.

4. Spring Scopes

Correct

Multiple Choice | Medium | Spring

Question description

Which of the following statements is/are true about @Scope("request") and @Scope("prototype") annotations?

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [x] Both are used to define the scope of spring beans.
- [x] @Scope("request") creates a bean instance for a single HTTP request.
- [x] @RequestScope can be used instead of @Scope("request").
- [x] @Scope("prototype") will return a different instance every time it is requested from the container.

No comments.

5. Predict the Output

Correct

Multiple Choice | Medium | Exception Handling | Spring Boot | Aspect-Oriented Programming

Question description

Consider the following Snippet and predict the output.

The following API is triggered: http://localhost:8080/add/employee?eId=123&fName=Ravi&lastName=Shekhar

```
@Autowired
private EService eService;

@RequestMapping(value = "/add/employee", method = RequestMethod.GET)
public com.model.Employee add(@RequestParam("eId") String eId, @RequestParam("fName") Strin
g fName, @RequestParam("lastName") String lastName)
{
   return eService.createEmployee(eId, fName, lastName);
}

@Aspect
@Component
public class EServiceAspect
{
   @After(value = "execution(* com.random.service.EService.*(..)) && args(eId, fName, lastName)")
   public void afterAdvice(JoinPoint joinPoint, String eId, String fName, String lastName) {
      System.out.println("After method:" + joinPoint.getSignature());
      System.out.println("Creating Employee with first name - " + fName+ ", second name - " + lastNa
me+ " and id - " + eId);
   }
}
```

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ✓ After method : com.test.model.Employee addEmployee(String,String,String)
  Creating Employee with first name - Ravi , second name - Shekhar and id - 123

After method : com.test.model.Employee addEmployee(string,string,string) Creating Employee with first name - Ravi , second name - Shekhar and id - 123

After method : com.test.model.Employee addEmployee(empId,firstName,secondName) Creating Employee with first name - Ravi , second name - Shekhar and id - 123

After method : com.test.model.Employee addEmployee(String,String,String) Creating Employee with first name - Shekhar, second name - Ravi and id - 123

No comments.

6. Minimize Path Value

Correct

Coding Graphs Medium

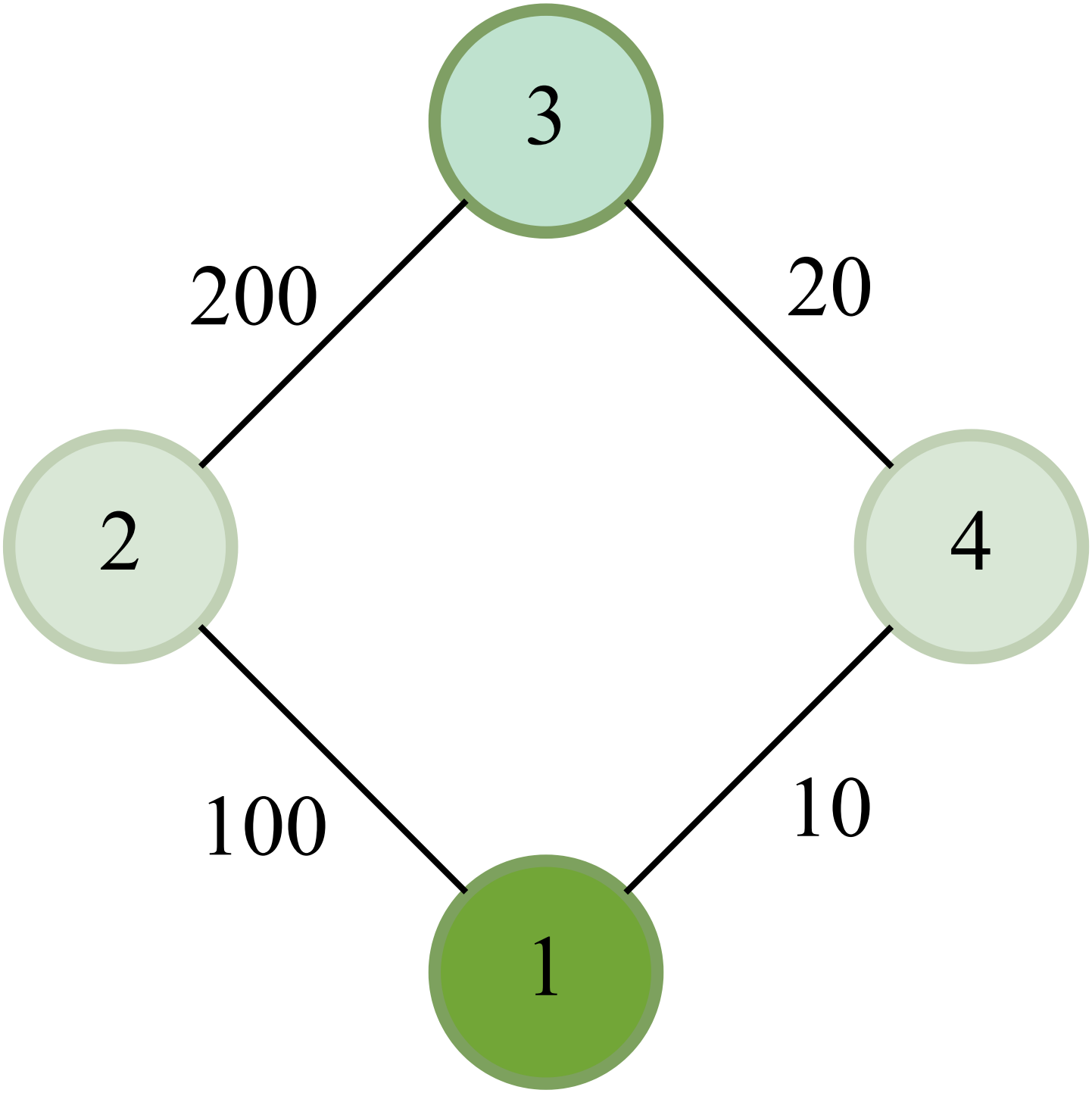
Question description

You have a weighted undirected graph with *graph_nodes* nodes and *m* edges. The stress level of a path between two nodes is defined as the weight of the heaviest edge in that path.

Given a source node *source* and a destination node *destination*, find the minimum possible stress level of a path. If no such path exists, return -1.

Example

Consider the following example below, where *source=1* and *destination=3.*



There are two paths, from node 1 to node 3.

1. The max weighted edge/stress level in path 1 -> 2 -> 3 is 200.
2. The max weighted edge/stress level in the path 1 -> 4 -> 3 is 20.

Return 20, the lower stress level from the second path.

Function Description

Complete the function *getMinimumStress* in the editor with the following parameter(s):

int graph_nodes: number of nodes
int graph_from[m]: one end node for an edge
int graph_to[m]: one end node for an edge
int graph_weight[m]: the weights of the edges
int source: the source node
int destination: the destination node

Returns

int: the value of the least stressful path, or -1 if no path exists

Note: Even though edge endpoints are called *graph_from[i]* and *graph_to[i]*, this is an undirected graph.

Constraints

- $1 \le graph_nodes \le 10^5$
- $1 \le |graph_from| = |graph_to| = |graph_weight| \le 10^5$ *(Where |A| denotes the size of array A)*
- $1 \le graph_from[i], graph_to[i], source, destination \le graph_nodes$
- $0 \le graph_weight[i] \le 10^9$

▼ INPUT FORMAT FOR CUSTOM TESTING

The first line contains two space-separated integers *graph_nodes* and *m,* the number of nodes, and the number of edges of the graph.
Each of the following *m* lines contains three space-separated integers *u, v,* and *w,* meaning there is an edge connecting the nodes *u* and *v* of weight *w.*
The next line contains a single integer, denoting the index of the "*source*" node.
The next line contains a single integer, denoting the index of the "*destination*" node.

▼ SAMPLE CASE 0

Sample Input For Custom Testing

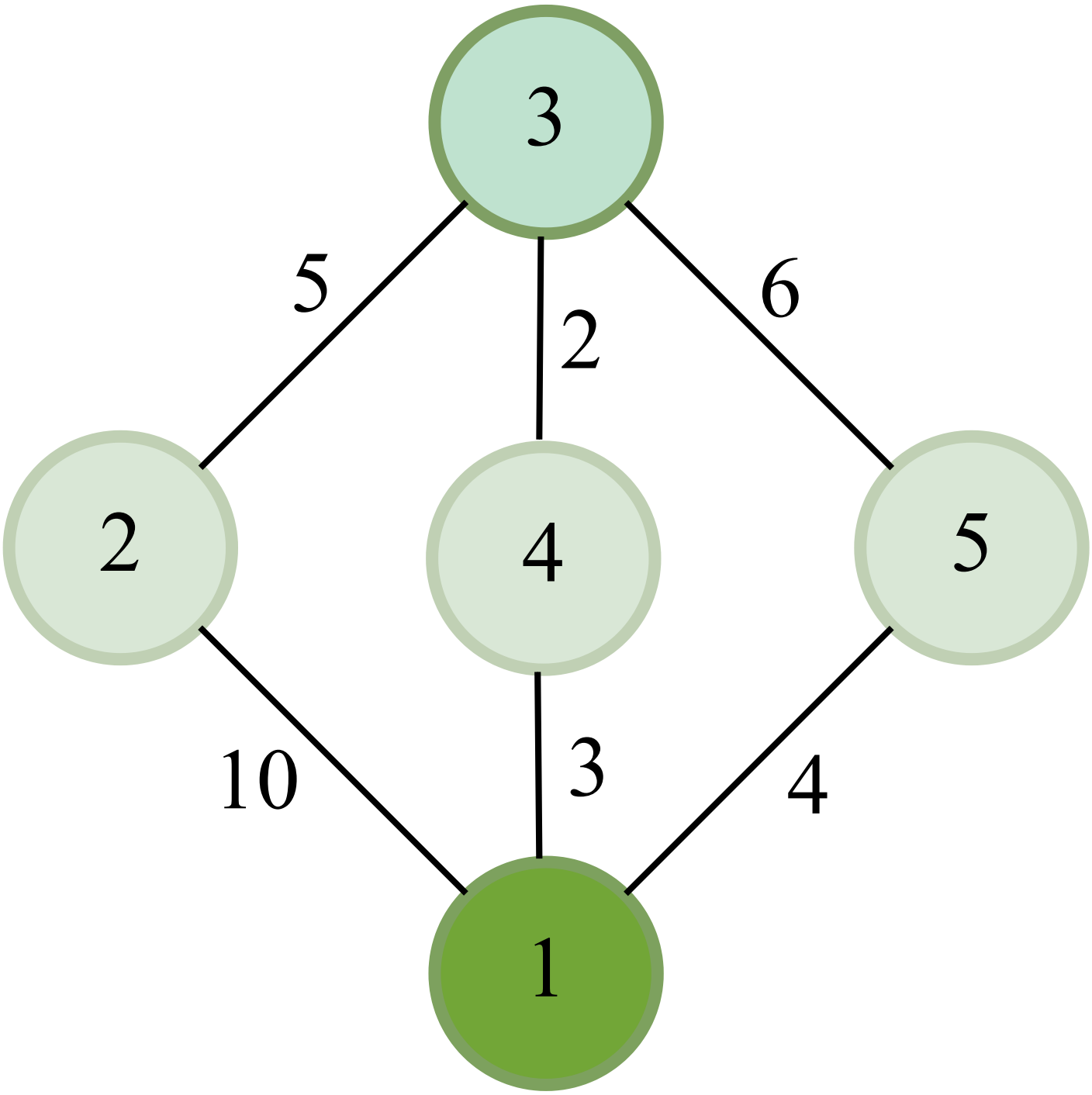
```
STDIN       FUNCTION
-----       --------
5 6         graph_nodes = 5, m = 6
1 2 10      u = 1, v = 2, w = 10
```

```
2 3 5
1 4 3
4 3 2
1 5 4
5 3 6
1          source = 1
3          destination = 3
```

Sample Output

```
3
```

Explanation



There are three paths, from node 1 to node 3 :

1. 1 -> 2 -> 3. Edges have weights 10 and 5, the maximum weighted edge, and the stress level is 10.
2. 1 -> 4 -> 3. Edges have weights 3 and 2, and the stress level = 3.
3. 1 -> 5 -> 3. Edges have weights 4 and 6, and the stress level = 6.

▼ SAMPLE CASE 1

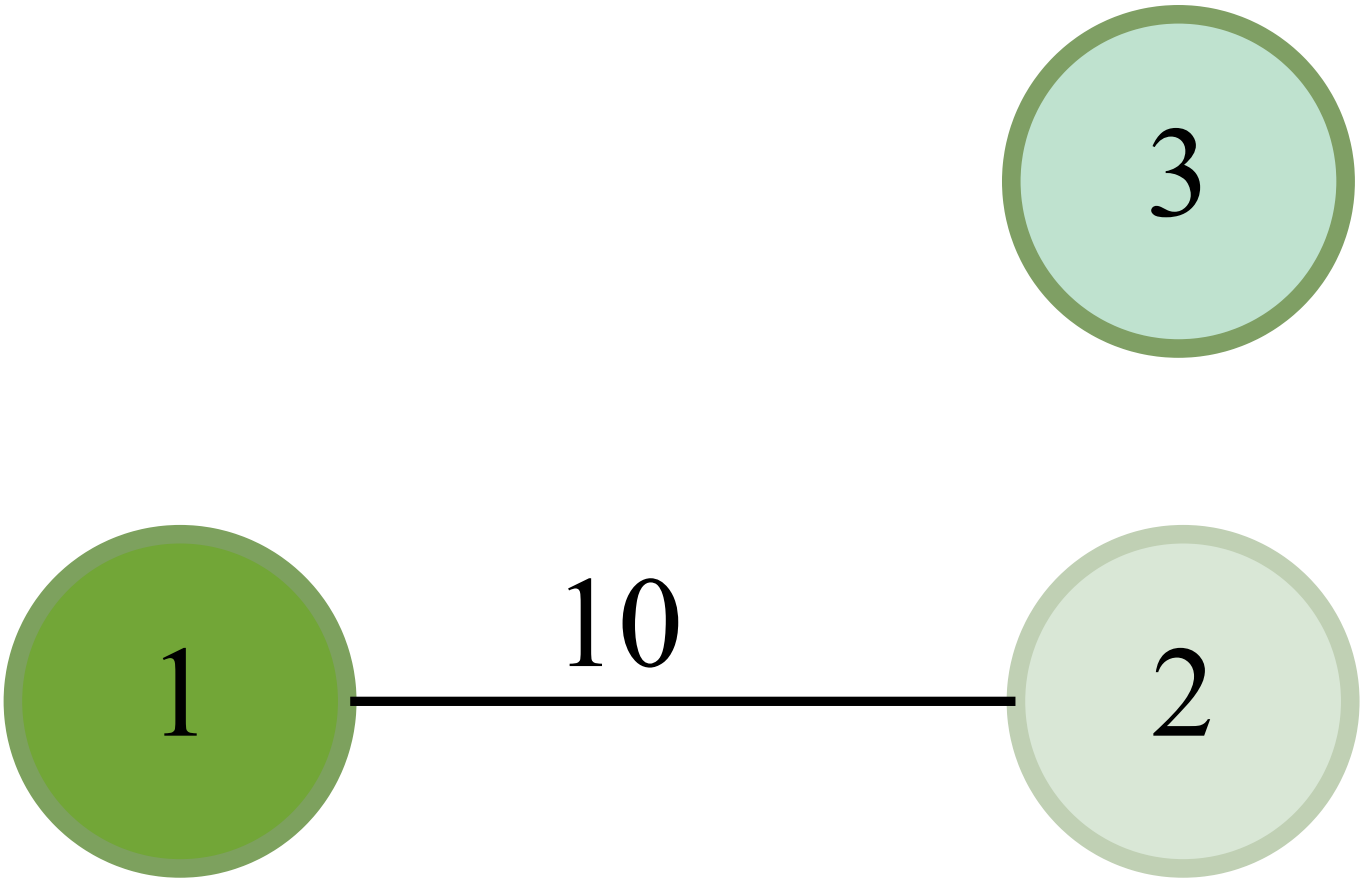
Sample Input For Custom Testing

```
STDIN       FUNCTION
-----       --------
3 1          graph_nodes = 3, m = 1
1 2 10       a = 1, b = 2, w = 10
1           source = 1
3           destination = 3
```

Sample Output

```
-1
```

Explanation



There is no path, from node 1 to node 3.

Interviewer guidelines

▼ SOLUTION

Python Solution

```
sys.setrecursionlimit(10**5)


def getMinimumStress(graph_nodes, graph_from, graph_to, graph_weight, source, destination):

    adj = [[] for i in range(graph_nodes + 1)]
    vis = [False] * (graph_nodes + 1)

    for x, y, w in zip(graph_from, graph_to, graph_weight):
        adj[x].append((y, w))
        adj[y].append((x, w))

    def dfs(node, mx):
        vis[node] = True
        if node == destination:
            return True
        ans = False
        for i, w in adj[node]:
            if w <= mx and not vis[i]:
                ans |= dfs(i, mx)
        return ans

    if not dfs(source, max(graph_weight)):
        return -1

    lo, hi = 0, max(graph_weight)
    while hi - lo > 1:
        vis = [False] * (graph_nodes + 1)
        mid = (hi + lo) // 2
        if dfs(source, mid):
            hi = mid
        else:
            lo = mid
    return hi
```

Candidate's Solution

Language used: **Java 8**

```
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
```

```java
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;


class Result {

    /*
     * Complete the 'getMinimumStress' function below.
     *
     * The function is expected to return an INTEGER.
     * The function accepts following parameters:
     *  1. WEIGHTED_INTEGER_GRAPH graph
     *  2. INTEGER source
     *  3. INTEGER destination
     */

    /*
     * For the weighted graph, <name>:
     *
     * 1. The number of nodes is <name>Nodes.
     * 2. The number of edges is <name>Edges.
     * 3. An edge exists between <name>From[i] and <name>To[i]. The weight of the edge is <name>Weight[i].
     *
     */

     static class Edge implements Comparable<Edge>{
        int u, v, weight;
        Edge(int u, int v, int weight){
            this.u = u;
            this.v = v;
            this.weight = weight;
        }

        @Override
        public int compareTo(Edge other){
            return Integer.compare(this.weight, other.weight);
        }
     }

     static class DSU{
        int[] parent;
```

```
        DSU(int n){
            parent = new int[n+1];
            for(int i=0;i<=n;i++) parent[i] = i;
        }
        int find(int i){
            if(parent[i] == i) return i;
            return parent[i] = find(parent[i]);
        }
        void union(int i, int j){
            int rootI = find(i);
            int rootJ = find(j);
            if(rootI != rootJ) parent[rootI] = rootJ;
        }
     }

    public static int getMinimumStress(int graphNodes, List<Integer> graphFrom, List<Integer> graphTo, List<Integer> graphWeight, int source, int destination) {
        if(source == destination) return 0;

        int m = graphFrom.size();
        List<Edge> edges = new ArrayList<>();
        for(int i=0;i<m;i++){
            edges.add(new Edge(graphFrom.get(i), graphTo.get(i), graphWeight.get(i)));
        }

        Collections.sort(edges);

        DSU dsu = new DSU(graphNodes);

        for(Edge edge: edges){
            dsu.union(edge.u, edge.v);
            if(dsu.find(source) == dsu.find(destination)){
                return edge.weight;
            }
        }

        return -1;
    }

}

public class Solution {
    public static void main(String[] args) throws IOException {
```

```
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));

        String[] graphNodesEdges = bufferedReader.readLine().replaceAll("\\s+$", "").split(" ");

        int graphNodes = Integer.parseInt(graphNodesEdges[0]);
        int graphEdges = Integer.parseInt(graphNodesEdges[1]);

        List<Integer> graphFrom = new ArrayList<>();
        List<Integer> graphTo = new ArrayList<>();
        List<Integer> graphWeight = new ArrayList<>();

        IntStream.range(0, graphEdges).forEach(i -> {
            try {
                String[] graphFromToWeight = bufferedReader.readLine().replaceAll("\\s+$", "").split(" ");

                graphFrom.add(Integer.parseInt(graphFromToWeight[0]));
                graphTo.add(Integer.parseInt(graphFromToWeight[1]));
                graphWeight.add(Integer.parseInt(graphFromToWeight[2]));
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        });

        int source = Integer.parseInt(bufferedReader.readLine().trim());

        int destination = Integer.parseInt(bufferedReader.readLine().trim());

        int result = Result.getMinimumStress(graphNodes, graphFrom, graphTo, graphWeight, source, destination);

        bufferedWriter.write(String.valueOf(result));
        bufferedWriter.newLine();

        bufferedReader.close();
        bufferedWriter.close();
    }
}

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
TestCase 0	Easy	Sample	Success	1	0.1319 sec	30.9 KB
TestCase 1	Easy	Sample	Success	1	0.1232 sec	31.4 KB
TestCase 2	Easy	Sample	Success	1	0.1118 sec	31.4 KB
TestCase 3	Easy	Hidden	Success	6	0.5199 sec	140 KB
TestCase 4	Easy	Hidden	Success	6	0.457 sec	133 KB
TestCase 5	Easy	Hidden	Success	6	0.5215 sec	135 KB
TestCase 6	Easy	Hidden	Success	6	0.6053 sec	150 KB
TestCase 7	Easy	Hidden	Success	6	0.4905 sec	151 KB
TestCase 8	Easy	Hidden	Success	6	0.6754 sec	139 KB
TestCase 9	Easy	Hidden	Success	6	0.6535 sec	149 KB
TestCase 10	Easy	Hidden	Success	6	0.5523 sec	144 KB
TestCase 11	Easy	Hidden	Success	6	0.5077 sec	138 KB

TestCase 12	Easy	Hidden	Success	6	0.4473 sec	78.7 KB
TestCase 13	Easy	Hidden	Success	6	0.4948 sec	96.6 KB
TestCase 14	Easy	Hidden	Success	6	0.7476 sec	150 KB

No comments.

7. Java: Optimized Sequential Video Watching

Correct

Coding Medium Greedy Java

Question description

A learner is given a sequential playlist of *n* videos to enhance their knowledge. Implement a function to calculate the minimum time required to complete a total of *m* viewings while adhering to the platform's constraints.

System Constraints

- Sequential Viewing:
  - The learner must watch videos in the order they appear in the playlist.
  - To watch a video at index *i*, the learner must first watch all videos from index 0 to *i-1* at least once.
- Initial Full Viewing:
  - To view any video at index *i* for the first time, the learner must spend *firstWatch[i] + repeatWatch[i]* minutes. This ensures a complete understanding of the video.
- Subsequent Viewings:
  - For subsequent viewings of a video at index *i*, the learner spends only *repeatWatch[i]* minutes.
  - At any time, they can rewatch any video they have already watched.

- Minimum Total Watch Count:
  - The learner must achieve at least *m* total viewings across all videos.
  - Videos can be rewatched as desired, and not all videos need to be watched equally.

Return the minimum time required to complete *m* viewings of videos while following these constraints.

Example

If $n = 4$, $m = 4$, *firstWatch* = [1, 5, 9, 11], and *repeatWatch* = [2, 7, 10, 11], the optimal strategy is as follows:

- The learner watches the 0th video for the first time, spending *firstWatch*[0] + *repeatWatch*[0] = 1 + 2 = 3 minutes.
- The learner then rewatches the 0th video 3 more times in, spending *repeatWatch*[0]*3 = 2 * 3 = 6 minutes.
- The learner has successfully completed 4 viewings. Thus, the total time required is 3 + 6 = 9 minutes, and 9 is returned.

Function Description

Complete the function *findMinMinutes* in the editor below.

Function Parameters

int m: the total number of video viewings to complete

int firstWatch[n]: the minutes required to watch each video the first time

int repeatWatch[n]: the minutes required to rewatch each video

Returns

long: the minimum minutes required for the learner to reach their target number of viewings

Constraints

- $1 \le n \le 10^5$
- $1 \le m \le 10^9$
- $1 \le firstWatch[i], repeatWatch[i] \le 10^9$

▼ INPUT FORMAT FOR CUSTOM TESTING

The first line contains an integer, *m*.

The next line contains an integer, *n,* the size of *firstWatch*.

Each of the next *n* lines contains an integer, *firstWatch[i]*.

The next line contains an integer, *n,* the size of *repeatWatch*.

Each of the next *n* lines contains an integer, *repeatWatch[i]*.

▼ SAMPLE CASE 0

Sample Input 0

```
STDIN    FUNCTION
-----    --------
5        m = 5
3        firstWatch[] size n = 3
6        firstWatch = [6, 8, 9]
8
9
3        repeatWatch[] size n = 3
4        repeatWatch = [4, 2, 10]
2
10
```

Sample Output 0

```
26
```

Explanation

The goal is to complete 5 viewings. One optimal strategy is:

- The learner watches the 0th video for the first time, so the time required is *firstWatch*[0] + *repeatWatch*[0] = 6 + 4 = 10 minutes.
- The learner watches the 1st video for the first time, so the time required is *firstWatch*[1] + *repeatWatch*[1] = 8 + 2 = 10 minutes.
- The learner then rewatches the 1st video 3 times. So the time taken = *repeatWatch*[1]*3 = 2 * 3 = 6 minutes.
- The total time required to complete 5 video viewings is 10 + 10 + 6 = 26 minutes.

▼ SAMPLE CASE 1

Sample Input 1

```
STDIN     FUNCTION
-----     --------
6         m = 6
5         firstWatch[] size n = 5
2         firstWatch = [2, 3, 7, 8, 4]
3
7
```

```
8
4
5       repeatWatch[] size n = 5
3       repeatWatch = [3, 6, 8, 7, 6]
6
8
7
6
```

Sample Output 1

```
20
```

Explanation

The goal is to complete 6 viewings. One optimal strategy is:

- The learner watches the 0th video for the first time in *firstWatch*[0] + *repeatWatch[0]* = 2 + 3 = 5 minutes.
- The learner then rewatches the 0th video 5 times in *repeatWatch[0]**5 = 3 * 5 = 15 minutes.
- The total time required to complete 6 viewings is 5 + 15 = 20 minutes.

Interviewer guidelines

▼ SOLUTION

Description

The greedy solution suggests that, at any given index, if the time to watch the video is less than at any other index between i+1 and n, the minimum total time will be the sum of the watch time up to index i-1, plus the repeat time of the ith video multiplied by (m-i).

Solution

```
class Result {

    /*
     * Complete the 'findMinMinutes' function below.
     *
     * The function is expected to return a LONG_INTEGER.
     * The function accepts following parameters:
     *  1. INTEGER m
     *  2. INTEGER_ARRAY firstWatch
     *  3. INTEGER_ARRAY repeatWatch
     */

    public static long findMinMinutes(int m, List<Integer> firstWatch, List<Integer> repeatWatch) {
```

```
        long minMinutes = Long.MAX_VALUE;
        long cur = 0;
        int n = firstWatch.size();

        for (int i = 0; i < Math.min(n,m); i++) {
            int a = firstWatch.get(i);
            int b = repeatWatch.get(i);

            minMinutes = Math.min(minMinutes, cur + ((long) (m - i) * b) + a);

            cur += a + b;
        }
        return minMinutes;
    }

}
```

▼ COMPLEXITY

Time Complexity - O(n)

The algorithm iterates through each number in *firstWatch* and *repeatWatch* exactly once, performing constant-time checks and arithmetic operations in each iteration. Thus, the time complexity is O(n), where *n* is the length of *firstWatch*.

Space Complexity - O(1)

Space usage is constant, only utilizing a fixed number of variables (*mn, cur,* etc.) that do not scale with the length of *firstWatch*. Therefore, the space complexity is O(1).

Candidate's Solution

Language used: **Java 8**

```
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;

```

```java


class Result {

    /*
     * Complete the 'findMinMinutes' function below.
     *
     * The function is expected to return a LONG_INTEGER.
     * The function accepts following parameters:
     *  1. INTEGER m
     *  2. INTEGER_ARRAY firstWatch
     *  3. INTEGER_ARRAY repeatWatch
     */

    public static long findMinMinutes(int m, List<Integer> firstWatch, List<Integer> repeatWatch) {
            int n = firstWatch.size();
            long minTime = Long.MAX_VALUE;

            long unlockCost = 0;
            long minRepeat = Long.MAX_VALUE;

            for(int i=0;i<n && i<m;i++){
                unlockCost += (long) firstWatch.get(i)+repeatWatch.get(i);
                minRepeat = Math.min(minRepeat, repeatWatch.get(i));
                long viewsSoFar = i+1;
                if(viewsSoFar>= m){
                    minTime = Math.min(minTime, unlockCost);
                }else{
                    long remaining = m-viewsSoFar;
                    long totalTime = unlockCost+remaining*minRepeat;
                    minTime = Math.min(minTime, totalTime);
                }
            }
        return minTime;
    }

}

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));

```

```java
        int m = Integer.parseInt(bufferedReader.readLine().trim());

        int firstWatchCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> firstWatch = IntStream.range(0, firstWatchCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

        int repeatWatchCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> repeatWatch = IntStream.range(0, repeatWatchCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

        long result = Result.findMinMinutes(m, firstWatch, repeatWatch);

        bufferedWriter.write(String.valueOf(result));
        bufferedWriter.newLine();

        bufferedReader.close();
        bufferedWriter.close();
    }
}

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
Testcase 2	Easy	Sample	Success	1	0.135 sec	31.9 KB
Testcase 0	Easy	Sample	Success	1	0.1375 sec	31.8 KB
Testcase 1	Easy	Sample	Success	1	0.1259 sec	31.8 KB
Testcase 3	Easy	Hidden	Success	4	0.1117 sec	31.9 KB
Testcase 4	Easy	Hidden	Success	4	0.1121 sec	32.3 KB
Testcase 5	Easy	Hidden	Success	4	0.1815 sec	35.9 KB
Testcase 6	Medium	Hidden	Success	5	0.2937 sec	90.8 KB
Testcase 7	Medium	Hidden	Success	5	0.4743 sec	176 KB
Testcase 8	Medium	Hidden	Success	5	0.6061 sec	175 KB
Testcase 9	Medium	Hidden	Success	5	0.5049 sec	175 KB
Testcase 10	Medium	Hidden	Success	5	0.5409 sec	178 KB
Testcase 11	Medium	Hidden	Success	5	0.4631 sec	165 KB

Testcase 12	Hard	Hidden	Success	10	0.4999 sec	97.4 KB
Testcase 13	Hard	Hidden	Success	10	0.7353 sec	168 KB
Testcase 14	Hard	Hidden	Success	10	0.1298 sec	31.8 KB

No comments.